Collective Communication

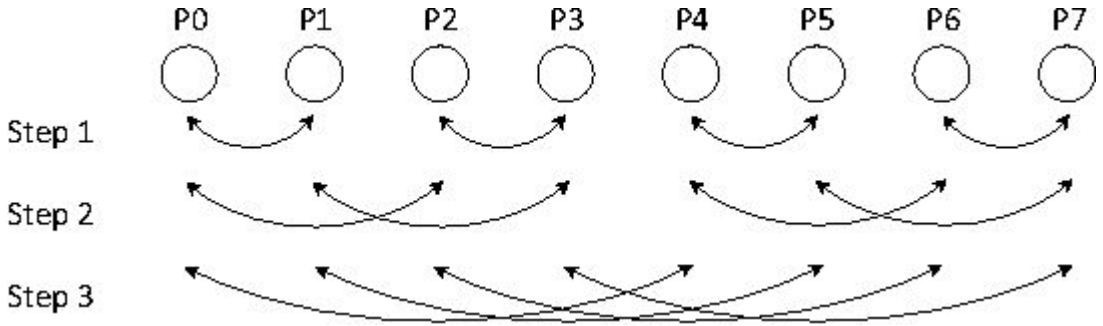
Collective Algorithms

- Multiple implementations exist for each MPI collective
- No single algorithm performs best in all situations
- Selection depends upon -

Message size, Number of processes (P) and Network characteristics (latency, bandwidth)

MPI_Allreduce: Recursive Doubling

- Processes exchange data with a partner in each round
- After each exchange, partial results are reduced
- Recursive Doubling is used for **small messages** because it requires only $\log_2(P)$ communication steps, minimizing latency, which dominates when message size is small.



MPI_Allreduce: Recursive Doubling for Non Power of 2

If P is not a power of two:

1. Extra processes are paired with neighbors
2. They send their data and become temporarily inactive
3. Remaining processes form the nearest power-of-two group
4. Recursive Doubling runs normally
5. Final result is sent back to the excluded processes

Example

P = 10

→ 2 processes excluded

→ Algorithm runs on P = 8

MPI_Allreduce: Recursive Halving and Doubling

Phase 1: **Reduce-scatter (recursive halving)**

At each step:

- Processes exchange **half of remaining data**
- Keep the half they are responsible for
- Reduce received data with local data

Step - 1

P0 : [A, B, C, D]
P1 : [E, F, G, H]
P2 : [I, J, K, L]
P3 : [M, N, O, P]

Step - 2 Reduce Scatter

P0 : [A+I, B+J]
P1 : [E+M, F+N]
P2 : [C+K, D+L]
P3 : [G+O, H+P]

Step - 3 Reduce-Scatter

P0 : [A+E+I+M] (sum0)
P1 : [B+F+J+N] (sum1)
P2 : [C+G+K+O] (sum2)
P3 : [D+H+L+P] (sum3)

Phase 2: **Allgather (recursive doubling)**

- Processes exchange the data they currently hold with a partner
- The amount of data **doubles** after each exchange
- Received data is **appended** to local data (no reduction)

Step - 4 Allgather

P0 : sum0, sum1
P1 : sum0, sum1
P2 : sum2, sum3
P3 : sum2, sum3

Step - 4 Allgather

All ranks: [sum0, sum1, sum2, sum3]

Used for medium size messages

MPI_Allreduce: Ring Allreduce

Processes are arranged in a logical ring.

Allreduce is performed in **two phases** and Each process communicates only with its **left and right neighbor**

1. **Reduce-Scatter (Ring)**
2. **Allgather (Ring)**

Phase 1: Reduce-Scatter (Ring)

At each step:

- Each process sends one data chunk to the **next process** in the ring
- Receives a chunk from the **previous process**
- **Reduces** the received chunk with its local chunk
- Number of steps = **P − 1**

Phase 2: Allgather (Ring)

At each step:

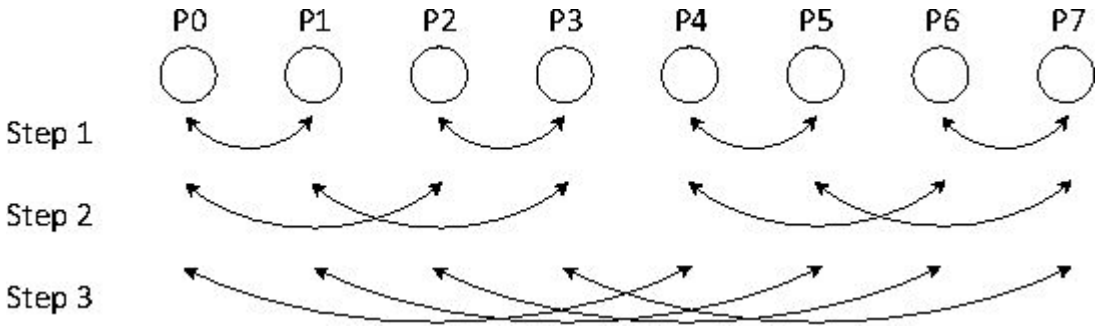
- Each process sends its current chunk to the **next process**
- Receives a new chunk from the **previous process**
- **Stores (no reduction)**
- Number of steps = **P − 1**

Best for large messages: ***Data is divided into chunks, and each chunk is forwarded immediately after being received, enabling continuous pipelined communication without waiting for the full message.***

MPI_Allgather: Recursive Doubling

At each step :

- Exchange **all currently owned data** with partner
- **Append** received data (no reduction)
- Data size **doubles** after each step
- Number of steps = $\log_2(P)$
- **Used for small messages.**



MPI_Allgather: Ring Allgather

Processes form a logical ring

At each step:

- Each process sends **one data chunk** to the **next process** in the ring
- Receives a **new chunk** from the **previous process**
- Stores the received chunk (**no reduction**)
- After each step, the amount of known data **increases by one chunk**
- Number of steps = **P − 1**

Best for large messages: *Data is divided into chunks and each chunk is forwarded step-by-step, enabling bandwidth-efficient communication.*

MPI_Gather and MPI_Reduce

- Linear Reduce/Gather
- Binomial Tree Reduce/Gather
- Pipelined Tree Reduce/Gather

Questions

1. Run MPI_Allreduce for different message sizes, plot execution time vs message size, and identify which algorithm (Recursive Doubling, Reduce-Scatter + Allgather, or Ring) performs best in the small, medium, and large message regions.
2. Run MPI_Allgather for varying message sizes, plot execution time vs message size, and identify which algorithm (Recursive Doubling or Ring) performs best for small and large message sizes.
3. Execute MPI_Reduce for multiple message sizes, plot execution time vs message size, and identify the message size ranges where Binomial Tree and Pipelined reduction perform best.
4. Execute MPI_Gather for varying message sizes, plot execution time vs message size, and identify the message size ranges where Linear, Binomial Tree, and Pipelined Gather perform best.
5. Run MPI_Reduce_scatter for different message sizes, plot execution time vs message size, and identify the message size ranges where recursive halving–based and large-message (chunked/pipelined) Reduce-Scatter algorithms perform best.
6. Repeat the experiment for a non-power-of-two number of processes, plot execution time vs message size, and compare the performance with the power-of-two case to identify the impact on algorithm behavior.

THANK YOU